CS 246 A: Artificial Intelligence and Decision Support

Instructor: Monika Stepanyan

Group Members: Milena Manukian, Robert Sarinyan, Robert Shakhnazarian, Armen Petrosyan

The 5th of December, 2025

LinkedIn Queens Game Project

Table of Contents

1. Introduction 3

2. Literature Review 5

2.1. Alternative Solving Approaches 6

3. Method 9

4. Evaluation 12

4.1 Accuracy and Run-time on Random Grids 12

4.2 Simulated Annealing Parameter Sensitivity 13

4.3 Runtime Distributions 14

4.4 Grid Complexity and Solver Behavior 17

5. Conclusion 21

References 25

1. Introduction

The LinkedIn Queens puzzle is a modern variation of the classic N-Queens puzzle. In the classical formulation, the task is to place the queens so that none attack each other horizontally, vertically, or diagonally. LinkedIn's adaptation extends the original puzzle by adding colored regions on the grid. It also modifies the "diagonal" constraint, restricting it only to diagonally adjacent positions of queens. Such adjustments transform the well-known Queens puzzle into a richer and a more sophisticated problem that requires exhaustive problem-solving and analysis.

The LinkedIn Queens problem is particularly suitable for the Artificial Intelligence course as it is visually simple yet structurally complex. While the grid layout is easily understandable, the interactions between the constraints might require deep search. By changing the size of the grid or reshaping the colored regions, the problem might become much more difficult to solve. Thus, this puzzle provides a chance to understand the transition from a popular game to a formally defined problem with its variables, domains, and constraints.

The Queens puzzle is a great example of a constraint satisfaction problem (CSP). The variables are the rows, whereas the domains are the set of available columns for a given row. The constraints state that exactly one queen can be placed per column or colored region, and besides, no two queens can be placed on adjacent (including diagonally adjacent) squares. By experimenting with 8x8 and 12x12 grid models, we can capture shifts in running time and completeness (ability to find a solution) of the algorithms and inference procedures described in coming chapters.

The rest of the report is as follows. In the Literature Review section, we discuss the previous approaches to solving the N-Queens problem as well as LinkedIn's variant of the puzzle. The Search Approaches section touches upon the algorithms we have selected for this project. The Method section describes in detail how we generated the grids and how each algorithm has been implemented. The Evaluation section presents and interprets our results, plots, and analyses. The Conclusions section connects the results to project goals and outlines possible directions for future work. The report closes with a list of references.

2. Literature Review

This chapter explores the history of the methods and techniques used to solve the general N-Queens problem and our LinkedIn Queens puzzle in particular. The focus is on previously published peer-reviewed articles that used classical approaches, as well as alternative ways to tackle the Queens puzzle. This chapter served as a starting point for us to understand which methods to implement based on their running time, success rate, and possibility to perform sufficient analyses with them.

Constraint Networks and Backtracking Algorithm

An important stage in history was the study of consistency in constraint networks. Mackworth (1977) defined a clean framework for arc and path consistencies and showed how they work with Backtracking Search to eliminate impossible options before even diving deeper into search and exploration. The core idea is that if a value is not supported by its neighbors, it should be deleted immediately. This establishes the basis for constraint propagation that remains critical in modern approaches for solving CSP-s.

MRV and LCV principles; AC-3 Inference Procedure

Soon after, Haralick and Elliott (1980) analyzed how search can be made practically more efficient through variable and value ordering. Two main techniques used are the fail-first/MRV and fail-last/LCV principles. Combined with simple Forward checking, these methods are capable of shrinking the search tree dramatically. Alongside these methods, enforcing arc-consistency using the AC-3 inference procedure modifies the variable domains easily Thus, AC-3 helps reduce the search space before Backtracking begins its execution (Bessière, 1994). These

ideas are relevant to our LinkedIn Queens puzzle, where the rows, columns, colored regions, and adjacency form tight interactions, requiring pruning and ordering to handle large search spaces.

Local Search

The Queens puzzle grid can be treated as a complete configuration (all the queens are already placed, one per row), with the objective of moving the queens to minimize conflicts according to the constraints. Common techniques are Hill-Climbing and Simulated Annealing, which are local search algorithms aimed at reducing row, column, region, and adjacency violations. Even though these methods are not complete, they can be fast on large grids because they "repair" a single state instead of exploring a tree with partial assignments.

2.1. Alternative Solving Approaches

Although our focus is on the classical methods, it is useful to know other approaches that apply well when solving the Queens problem:

Z3 Solver

One approach is to encode the Queens puzzle in a solver like *Z3*. Z3 is an SMT (satisfiability modulo theories) tool developed by Microsoft Research. It is often used to encode constraints and solve them logically. The constraints here are logical formulas, defining the conditions that must be satisfied. Z3 then continues the exploration of the space of possible assignments of positions and returns a model satisfying all the constraints, if such one exists. Thus, Z3 highlights the power of theorem solvers, particularly for the LinkedIn's Queens game, because they can alternatively find solutions to various puzzles without having to solve them manually. (Masri A., 2024, Aug. 16).

Linear Programming (LP)

Moreover, the LinkedIn Queens game has been modeled as a constraint-based optimization problem using Linear Programming (LP). The core idea here is that the squares are viewed as binary decision variables, where the value is 1 if a queen is placed on a square, and 0 otherwise. The goal of LP is to determine the optimal number for each decision variable and maximize the number of 1s, while maintaining that it does not exceed the total number of queens that should be placed on the grid. Also, we need to account for the uniqueness constraints, i.e., exactly one queen per row, column, region, and a 2x2 subsquare to prevent adjacency (Walker, 2024).

Comparison of the mentioned techniques

Another crucial step of studying currently available research about the Queens game is finding ways to potentially improve the existing solutions and understanding how various solutions are related to each other. The already mentioned Backtracking and Forward Checking provide a useful framework for analyzing the problem. However, alternative approaches such as local search algorithms, Z3, and Linear Programming demonstrate that the problem can be tackled in different ways, depending on its scalability and adaptability. By comparing such differing strategies, one can capture patterns under which specific inference procedures or heuristic-driven approaches work most effectively.

Elevating CSPs to solve real-world problems

Lastly, there has been an emerging tendency of elevating the CSP frameworks to be able to solve real-world issues in various domains, such as logistics, routing, and scheduling (Zhang et al., 2025). In those fields, the constraints are often very dynamic in size and partially uncertain. This introduces a need for new, more complex technologies with domain-specific heuristics to improve performance and solve the given problem. Thus, such a shift demonstrates that CSP-s, once studied only as a theoretical model, have now become practical tools for tackling new real-world problems with increasing complexity.

3. Method

To solve the LinkedIn-style Queens Puzzle, we devised a step-by-step method that can be easily recreated by other people. Our first task was to create a generator that makes valid and always-solvable puzzles. For each grid size NxN, we began by building a correct classical N-

Queens solution. In this solution, one safe column is provided for each row. Each of these queen positions was assigned a unique region number, and we then slowly expanded these regions into the surrounding cells until we had filled the entire NxN grid. During this expansion we enforced the condition that regions should remain connected to not lead to dead ends. If the expansion failed, the generator restarted and tried again. Through this process we got region layouts that were very similar in style and constraints to the puzzles seen on LinkedIn, and we also knew for sure that at least one valid placement of queens existed in each grid.

Once we generated the puzzles, we defined the Queens Game as a constraint satisfaction problem. Each row was treated as a variable with its domain as all possible column indices. The constraints were defined according to the rules of the puzzle: 1) no two queens can be in the same column, 2) the queens cannot be in the same region, and 3) the queens cannot be adjacent horizontally, vertically, and diagonally. This CSP formulation enabled us to carry out both the classical search approaches and more modern local search from a common, unified framework.

At this point, we were already ready to implement a number of solvers so that we could study and compare different approaches. We started from a simple Backtracking depth-first search that gives a row-by-row placement of queens, verifies all the constraints, and then proceeds. Because Backtracking alone does not use constraint propagation, we also implemented MAC using the inference procedure AC-3 and Forward checking after each assignment. This solver is more computationally expensive, yet it eliminates inconsistent values early enough to reduce the branching factor.

We then focused on local search based methods. We implemented the Min-conflicts algorithm which starts with a random placement of one queen per row and attempts to "repair"

such a placement rather than building solutions one row at a time. A state is represented as a complete assignment: an array where the entry at index *r* is the column of the queen in row *r*. In each iteration we first find the rows where queens are involved in at least one conflict. We pick one of these rows randomly and create child states by trying out all of the possible columns (for that row) for the queen. For each candidate column we move the queen temporarily, calculate the number of conflicts and keep only the columns with the minimum number of conflicts. The next state is reached by moving the queen to one of these minimum conflict columns chosen randomly. To make the search more reliable, we have also added a restart mechanism, so if the search does not find a solution within a given number of steps, they restart from a fresh random configuration.

Finally, we came up with a Simulated Annealing solver, which has a simpler neighborhood. In each step we propose one random child state by selecting one random row and moving the queen in one random column. If this move decreases (or maintains) the number of conflicts we always accept it. If it increases the conflicts, we accept it only with probability$exp(-\triangle / T)$, where $\triangle$ is the increase in conflicts and T is the current temperature after each move the temperature is multiplied by the cooling rate. This way, the algorithm is more willing to accept "bad" moves at the start of the search (to get out of local minima) and more conservative as the temperature falls.

In order to evaluate the solvers fairly we created fixed solvable 8x8 and 12x12 grids using known random seeds. Every solver was then tested against the same grids, so that the performance could be compared directly. For each run we recorded whether the solver was successful, as well as the average, minimum, and maximum time it took for the algorithm to find

a solution. We also studied how algorithm parameters affect performance by running Min-conflicts with varying numbers of restarts and Simulated Annealing with different initial temperatures and cooling rates. These parameter sweeps were then plotted to help us see the changes in the methods' performance under different settings.

4. Evaluation

In this section, we evaluate the five implemented methods - simple Backtracking, MAC with Forward Checking and AC-3, Min-conflicts, Min-conflicts with Restarts, and Simulated Annealing - by several parameters. The assessment consists of four parts. First, we measure the accuracy and running time of each solver on randomly generated 8x8 and 12x12 grids. Secondly, we analyze how solver parameters affect performance, paying special attention to the cooling

schedule during Simulated Annealing and the number of restarts with Min-conflicts. Third, we study the empirical distribution of execution time by running solvers on hundreds of fixed grids and studying their variability. Finally, we relate the solver's performance to a grid complexity metric designed to determine how the structural properties of the board affect the complexity of the problem.

4.1 Accuracy and Run-time on Random Grids

The first part of the evaluation examines how each solver works on a set of randomly generated 8x8 and 12x12 solvable grids. For each grid, all solvers are performed independently, and we record whether they found the right solution and how long it took them. Only successful launches are included in the results so that unsuccessful local search attempts do not lead to a distortion of the average execution time.

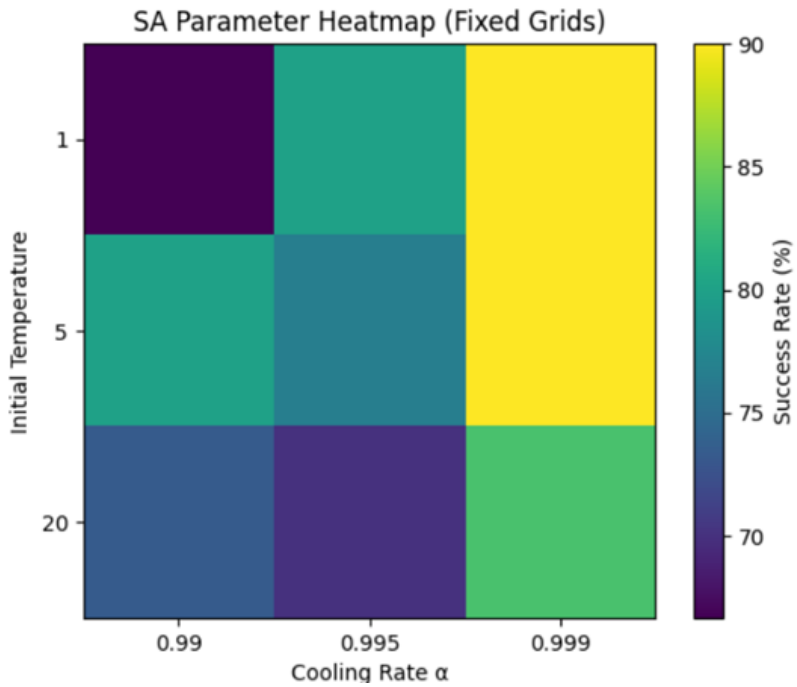
On 8x8 grids, the Backtracking method is extremely fast, while the MAC version is noticeably slower, since the additional output leads to significant overhead compared to the small size of the task. Min-conflicts without restarts successfully performed on only 66% of the grids but was extremely quick when it managed to find a solution. System failures occur when it gets trapped in local minima. Adding a small number of re-runs eliminates these failures and ensures near-perfect accuracy with minimal execution time. Simulated Annealing solves most problems but shows high variability during execution due to its probabilistic behavior.

On 12x12 grids, the contrast between the methods becomes more obvious. The Backtracking search remains completely reliable, although in some cases it takes significantly longer. The MAC method also solves all the problems, but it becomes the slowest of all solutions because it introduces computational overhead of enforcing the arc consistencies. Without

restarts, Min-conflicts now solved just over half of the grids, reflecting an increase in the number of local minima. However, when restarting, a perfect result is achieved, although the execution time depends on how many restart cycles are required. Simulated Annealing continues to work moderately well, but its effectiveness is dependent on our parameters choices, where highest cooling rates and low temperature produce the highest success rates.

4.2 Simulated Annealing Parameter Sensitivity

To better understand why Simulated annealing sometimes fails, the notebook includes an experiment with specific parameters on 12x12 boards. Our function first creates a fixed set of solvable grids so that the performance differences are due to the algorithm rather than random variations in the puzzles themselves. Then, Simulated annealing is performed using several combinations of the initial temperature (T0) and the cooling rate (alpha). For each pair, the solver is applied to all grids, and the percentage of successful runs is recorded. In our experiments, the best results were around a relatively low initial temperature ($T0 \approx 1$) and a moderately fast cooling (α closer to 0.99), while very high T0 or too slow cooling led to lower success rates. The results are displayed on a heatmap where the axes represent the two parameters and the color shows the probability of success.

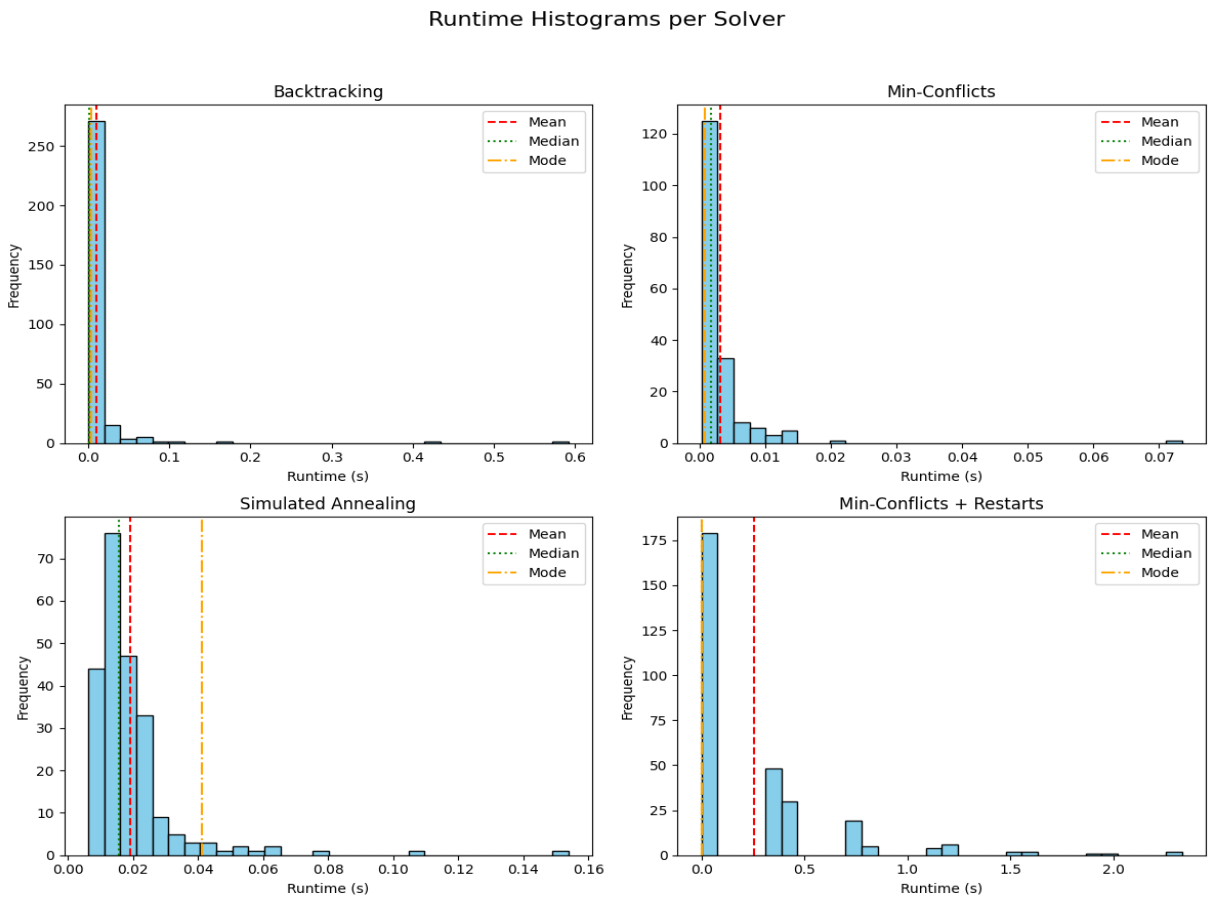


The heatmap portrays a clear trend. Lowering the temperature and increasing the cooling rate improves productivity. For example, when T0 = 1 and alpha = 0.99, the algorithm shows a high success rate. However, again, when the initial temperature is too high or the cooling rate is low, the success rate decreases, resulting in slow convergence of Simulated Annealing. In such cases, the solver would spend too much time exploring the state space rather than settling for more optimal solutions.

4.3 Run-time Distributions

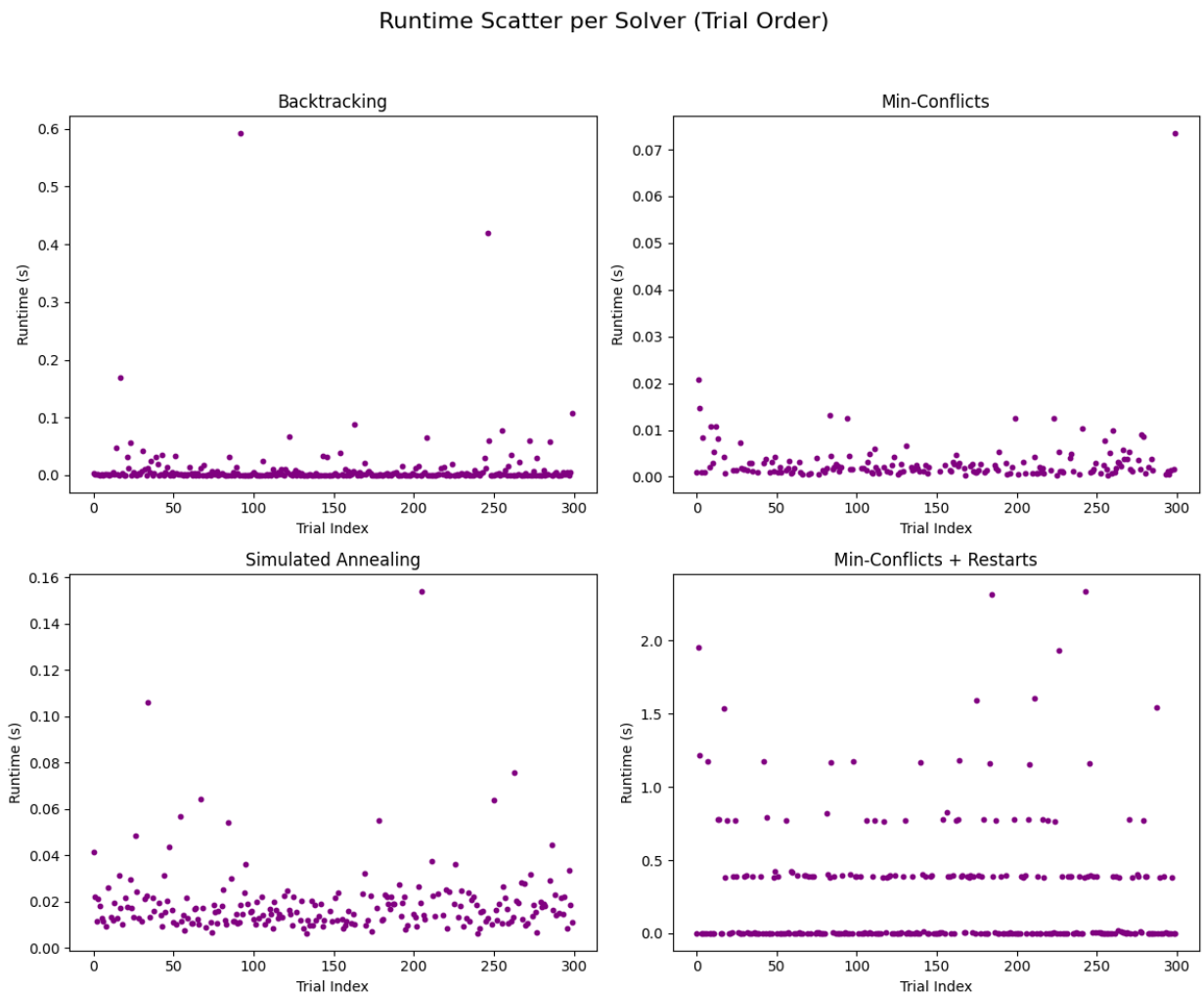
To better understand the computational behavior of the solvers, we conducted an additional experiment on 300 fixed 12x12 grids. For each solver, we measured the execution time for all successful runs and visualized the results using run-time histograms, scatter plots, and comparison boxplots.

The histograms show that the Backtracking search has a sharp peak with a very short running time, meaning it usually takes little time to solve the Queens puzzle. The Min-conflicts method is even closer to zero, reflecting extremely fast solutions when heuristic methods are successfully adapted. Simulated annealing shows a wider variation, mostly with short run-time bins, but with bins indicating longer run-time. Min-conflicts with restarts reveal the widest variation pattern: many fast solutions found, but also a long tail with bins exceeding two seconds. This highlights that, in some cases, several restarts were required to reach a solution.



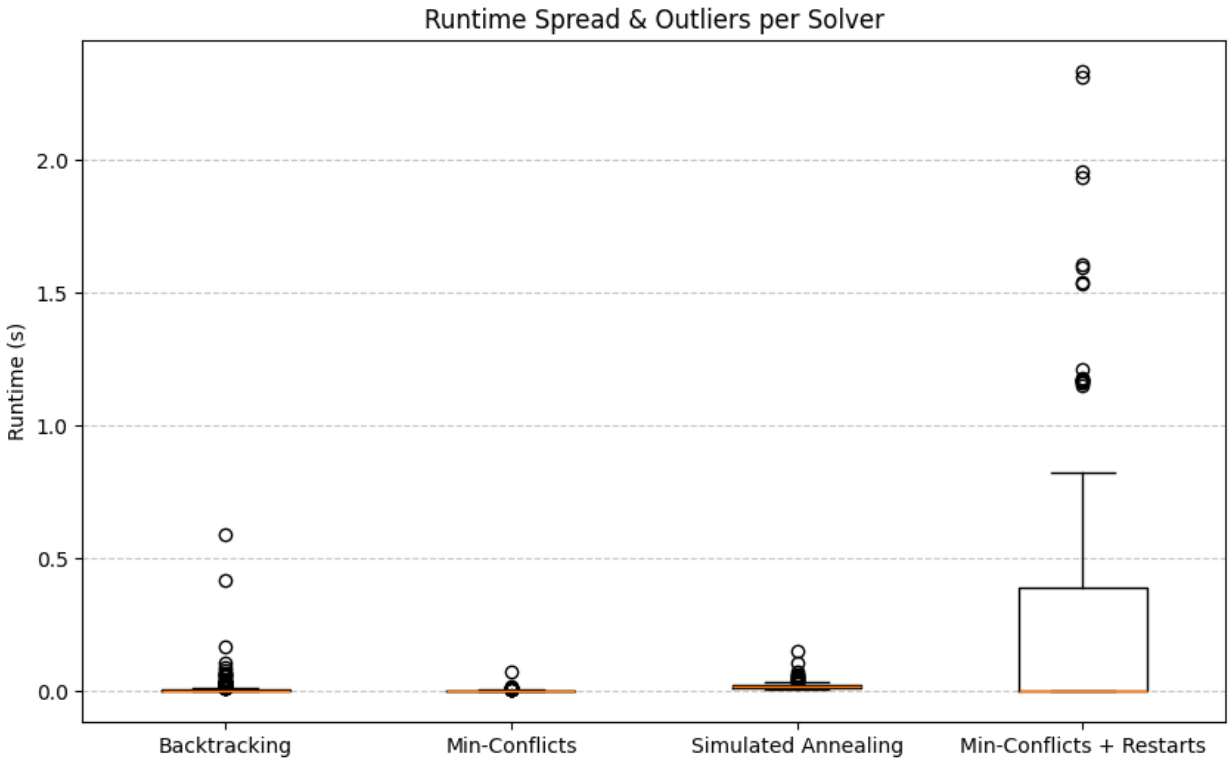
The Scatter plots provide additional insights. The time required for testing is distributed fairly across all trials, which suggests that the complexity mainly depends on the grid rather than the order of execution. Min-conflicts shows a similar pattern, but with consistently lower values.

During Simulated Annealing, stronger fluctuations (heteroskedasticity) are observed, which is likely caused due to the algorithm getting stuck in local minima. In Min-conflicts with restarts, there are dense clusters near zero and occasional high bursts near 0.5 run-time, which can be explained by the behavior of the "long tail" observed in the histograms.



The boxplot diagram is then plotted for evaluating the run-time spread and outliers for each solver. For Backtracking, a small compact IQR (Interquartile range) with a small number of outliers is shown. Min-conflicts and Simulated annealing illustrate on average faster times to find

a solution, with fewer outliers. Min-conflicts with restarts has the highest median and the widest spread, reflecting the additional time required to reach a solution during repeated restart cycles.



4.4 Grid Complexity and Solver Behavior

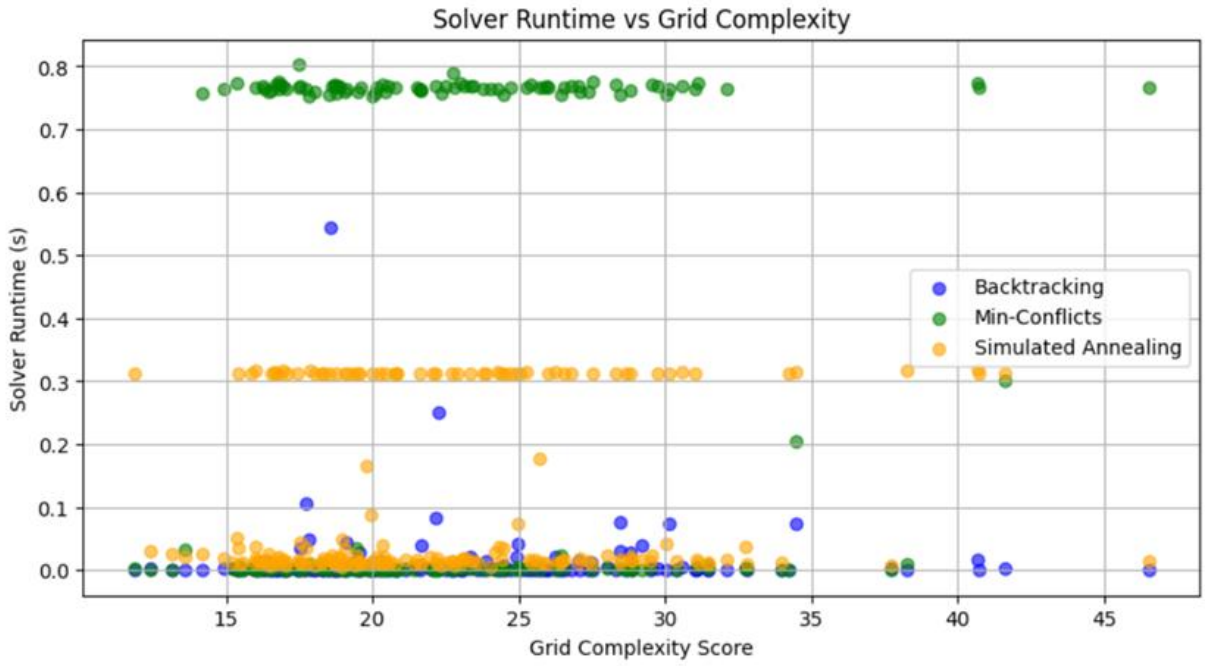
The final stage of the evaluation reveals how the grid structure influences our solvers' performance. Each 12×12 grid is assigned a complexity score based on three factors:

- **Total conflicts** when we randomly place one queen in each row
- **Region adjacency density** – how often cells in the same region touch each other in their immediate neighborhood (how "tight" the regions are)
- **Variance of region sizes** – how unevenly the board's squares are distributed across regions.

Higher scores correspond to structurally more complicated grids.

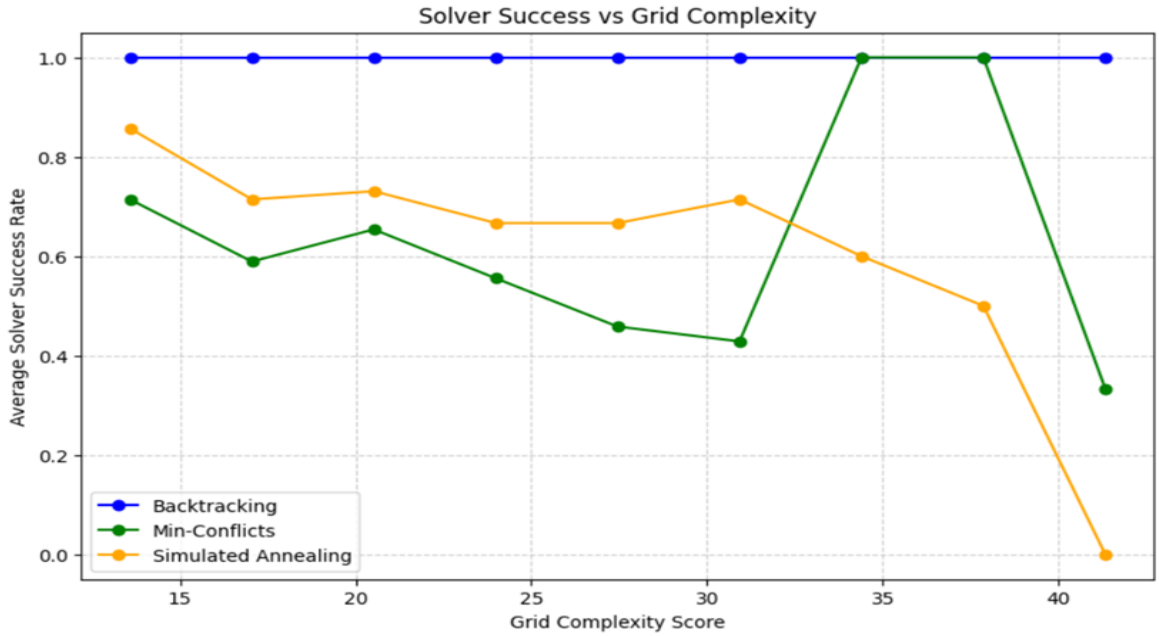
The function creates 200 grids using this metric, determines their complexity, and tracks each solver's execution time and success rate. Two graphs below clearly show the results: a success-rate plot categorized by difficulty ranges and a scatter plot showing run-time versus complexity.

The runtime graph demonstrates that Backtracking's complexity does not really change by varying execution time; it fluctuates around some fixed run-time values but lacks an easily interpretable pattern. On average, Min-conflicts and Simulated Annealing take longer time compared to Backtracking, because these grids with increasing complexity might require more restarts or deeper search.



The success-rate curves emphasize this pattern in a more clear way. Backtracking solves all problems regardless of their complexity. Min-conflicts works rather well on simple grids, but

its performance accuracy decreases quickly as complexity rises. Because of the probabilistic assignment of the “bad” (yet, helpful) moves, Simulated Annealing also shows a negative trend for average success rate as the complexity grows higher.



5. Conclusion

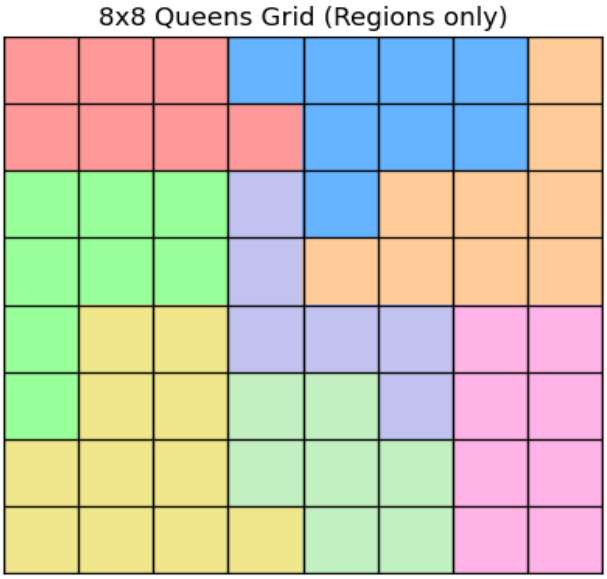
In this project, we examined methods for solving the LinkedIn Queens puzzle, a modern adaptation of the classical N-Queens problem. This grid-based puzzle, with its constraints, provided a compelling setting for research on constraint satisfaction problems (CSPs). Through the comparison of algorithms like Backtracking, Min-conflicts, and Simulated Annealing, and with their improved versions (MAC with Forward Checking and AC-3), we learned a lot about the performance of each algorithm, its strengths and weaknesses based on their use on different grid sizes.

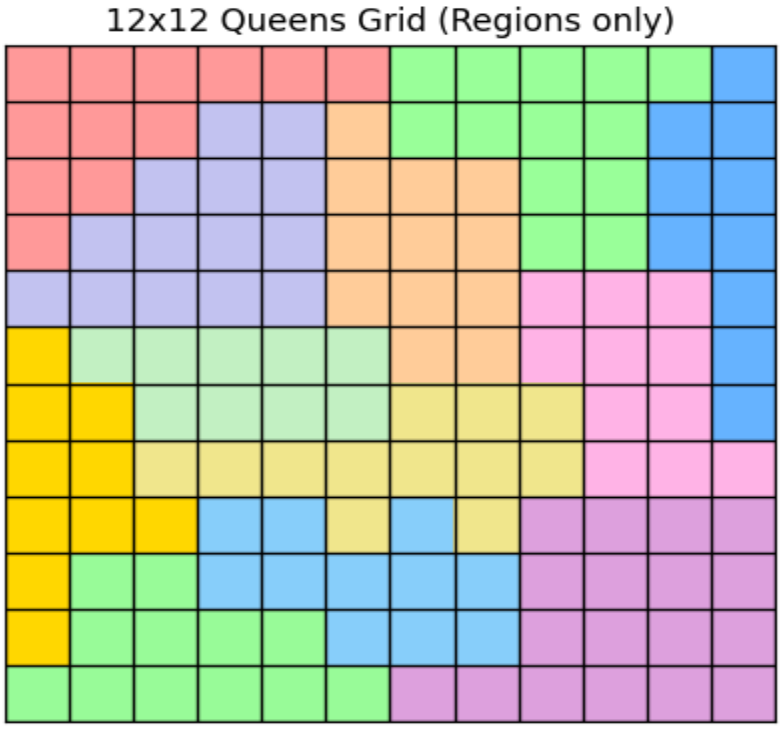
According to our findings, Backtracking is the most credible approach, but it might get computationally costly and slower with the complexity of the grid. Min-conflicts, considered on its own or modified to include restarts, is an efficient algorithm, although it fails at escaping local minima in the absence of restarts. Although effective and versatile, Simulated Annealing has a significant range of performance depending on the choice of parameters, most notably the temperature and the rate of cooling down. In addition, the parameter sensitivity analysis of the Simulated Annealing showed that a small initial temperature and a high cooling rate contribute to better results.

When applied together with Backtracking, the AC-3 strategy and Forward Checking strategy are considerably more efficient, pruning inconsistent values at earlier stages, but at the price of extra running time. Our findings reveal that the combination of iterative and local search techniques can offer a balanced solution, in which the iterative search methods can be used to ensure stability and the local methods can maintain high speed.

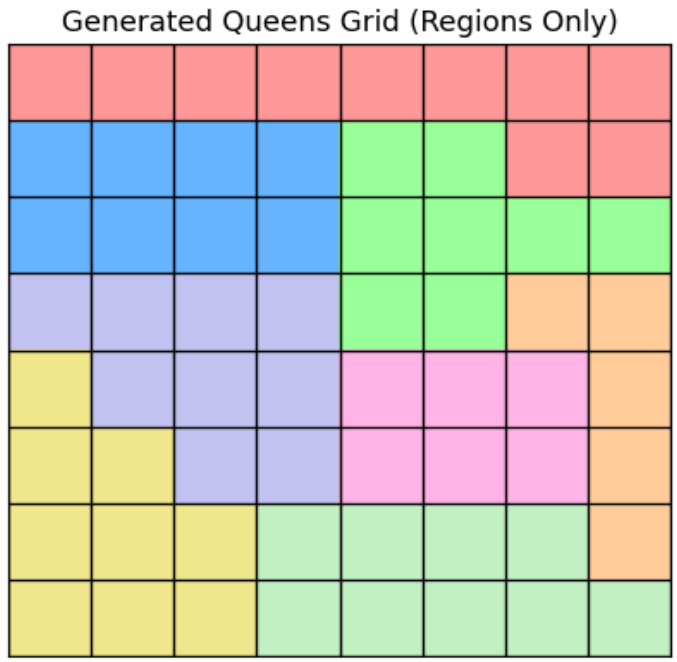
Future investigation may consider the use of more advanced approaches, including Machine Learning to tune parameters dynamically or using sophisticated local search methods to improve solution efficiency. Genetic Algorithm (GA) could also be incorporated to explore and handle large and complex solution spaces. Also, a further extension to the grid sizes and analysis of the real-world applications of CSPs, including in scheduling or logistics, would offer more context and endorsement of these techniques.

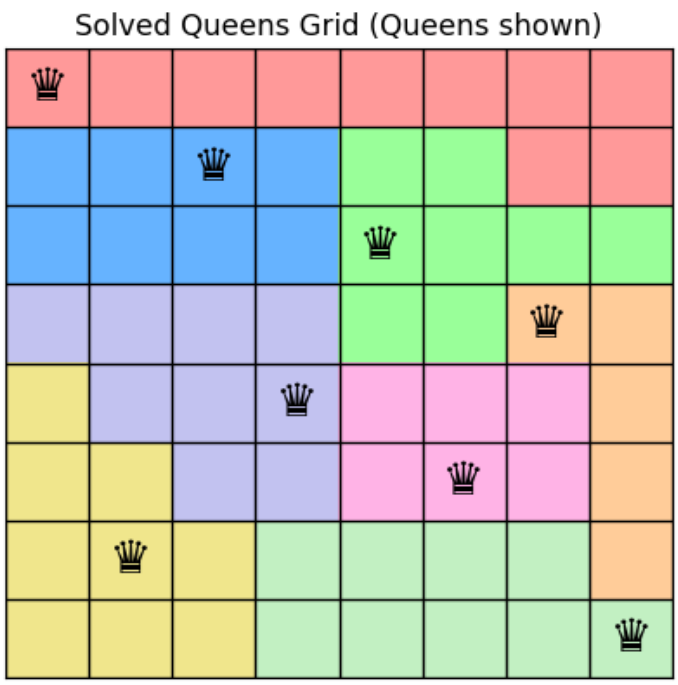
Examples of Solvable 8x8 and 12x12 Grids





Sample 8x8 Grid and Solution with Queens





References

Bessière, C. (1994). *Arc-consistency and arc-consistency again. Artificial Intelligence, 65*(1), 179–190. https://doi.org/10.1016/0004-3702(94)90041-8.

Haralick, R. M., & Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence, 14*(3), 263–313. (Fail-first/MRV intuition, look-ahead, ordering to shrink search). https://haralick.org/journals/haralick_elliot.pdf.

Walker, B. (2024, September 12). *Using linear programming to solve the LinkedIn Queens Game* [Blog post]. Gitconnected / LevelUp. https://levelup.gitconnected.com/using-linear-programming-to-solve-the-linkedin-queens-game-53a82df34d8d.

Mackworth, A. K. (1977). Consistency in networks of relations. *Artificial Intelligence, 8*(1), 99–118. (Foundational arc/Path consistency theory; basis for AC-3-style propagation). https://www.cs.ubc.ca/~mack/Publications/AI77.pdf.

Masri, A. (2024, Aug. 16). *Solving LinkedIn's Queens Puzzle with Code — Introduction to Z3 Library*. *Towards Data Engineering*. (SMT/Z3 encoding of the LinkedIn variant; confirms one-per-row/column/region + non-adjacency). https://medium.com/towards-data-engineering/solving-linkedins-queens-puzzle-with-code-6f1c3aa23a40.

Zhang, L., Maqrot, S., Mouysset, F., & Bortolaso, C. (2025). *A constraint satisfaction problems based scalable framework to address large-scale realistic scheduling and routing problems*. In *Proceedings of the 14th International Conference on Operations Research and Enterprise Systems (ICORES 2025)* (pp. 45–56). https://www.scitepress.org/Papers/2025/131096/131096.pdf.